

S2S-10: Migration Scripts

Series: S2S — SaaS to SaaS Migration | **Notebook:** 10 | **Created:** April 2026 | **Last Updated:** 09/28/2026

Overview

Reusable scripts for SaaS-to-SaaS configuration export. These scripts automate the Monaco download → SaaS Upgrade Assistant (SUA) packaging workflow, producing a `.tar.gz` archive ready for upload to the target tenant. Both Bash (macOS/Linux/WSL) and PowerShell (Windows) versions are provided.

Why scripts instead of raw `monaco download` ? The SaaS Upgrade Assistant on the target tenant requires a specific archive format (`.tar.gz` with `exportMetadata.json`). These scripts handle platform detection, Monaco binary download, checksum verification, temporary token creation, export, and SUA-compatible packaging in a single command.

Table of Contents

1. [SaaS Upgrade Assistant Format](#)
 2. [Monaco Configuration Export \(Bash\)](#)
 3. [Monaco Configuration Export \(PowerShell\)](#)
 4. [Usage Notes](#)
 5. [Post-Export Workflow](#)
-

Prerequisites

Requirement	Details
API Token	Token with <code>apiTokens.write</code> scope on the source tenant — the scripts use it to create, then revoke, a short-lived export token
Shell (Option A)	Bash (macOS, Linux, or WSL) with <code>curl</code> , <code>jq</code> , <code>shasum</code> , <code>tar</code>
Shell (Option B)	PowerShell 5.1+ on Windows 10/11 (includes <code>tar.exe</code> natively)
Network	HTTPS access to <code>github.com</code> (Monaco download) and source tenant URL

1. SaaS Upgrade Assistant Format

The SaaS Upgrade Assistant (SUA) on the target tenant accepts configuration imports in a specific archive format. Understanding this format is essential — the scripts below produce it automatically.

Archive Requirements

Requirement	Detail
Format	<code>.tar.gz</code> — not <code>.zip</code> (SUA rejects zip archives)
Metadata file	<code>exportMetadata.json</code> at the archive root
Export directory	<code>export/</code> subdirectory containing the Monaco download output

`exportMetadata.json` Structure

```
{
  "clusterUuid": "&lt;source-tenant-id&gt;",
  "productVersion": "1.305.0.20260331-000000",
  "monacoVersion": "2.30.0",
  "exportTimestamp": "&lt;unix-ms&gt;",
  "environments": [
    {
      "name": "&lt;source-tenant-id&gt;",
      "uuid": "&lt;source-tenant-id&gt;"
    }
  ]
}
```

Directory Layout Inside the Archive

```
configurationExport-YYYY-MM-DD_HH-MM-SS/
├─ exportMetadata.json
├─ export/
│   └─ project/
│       ├── alerting-profile/
│       ├── auto-tag/
│       ├── dashboard/
│       ├── management-zone/
│       ├── notification/
│       └─ ... (50+ config types)
```

Key fact: If you run `monaco download` manually and upload the raw output, SUA will reject it. The scripts below handle the packaging automatically.

2. Monaco Configuration Export (Bash)

Downloads Monaco, exports all configuration from the source tenant, and packages it in SaaS Upgrade Assistant format.

Deprecation — Dynatrace API 1.348 (pre-release; staged rollout planned from 09/22/2026). The API 1.348 changelog marks the whole `/apiTokens` endpoint family

deprecated — *"The following endpoints are deprecated"*, covering `POST /apiTokens` , `POST /apiTokens/lookup` and the per-token `GET / PUT / DELETE` . No successor is named. Deprecated endpoints keep working during the deprecation period, so both scripts below (which mint a temporary export token with `POST /api/v2/apiTokens`) remain the working path — re-check the [API 1.348 changelog \(DT docs\)](#) at GA before building new automation on that endpoint. Separately, once an environment opts into **Phase 3** of the upgrade to Latest Dynatrace, *"classic API token creation is disabled; all new integrations must use platform tokens"* ([Best practices for upgrading App Observability API endpoints \(DT docs\)](#)) — check your environment's upgrade phase before running these scripts, since each one mints a classic token.

Usage:

```
export ENV_TOKEN="dt0c01.XXXX..."
./monaco-export-migration.sh <source-tenant-id>

# Example
./monaco-export-migration.sh abc12345
```

Script:

```

#!/bin/bash
#
# Monaco Configuration Export for SaaS Upgrade Assistant
# Produces a .tar.gz archive compatible with the SUA on the target tenant.
#
# Usage: ENV_TOKEN=<api-token> ./monaco-export-migration.sh
# <tenantId>

set -euo pipefail

MONACO_VERSION="2.30.0" # pinned; check the releases page before reuse

# --- Argument validation ---
if [ $# -ne 1 ]; then
    echo "Usage: $0 <tenantId>"
    echo ""
    echo "Example: ENV_TOKEN=\$MY_TOKEN $0 abc12345"
    exit 1
fi

if [ -z "${ENV_TOKEN:-}" ]; then
    echo "Error: ENV_TOKEN environment variable is not set."
    echo "Create a token with apiTokens.write scope at:"
    echo "  https://$1.live.dynatrace.com/#settings/integration/apikeys"
    exit 1
fi

tenantId="$1"
echo "=== Monaco Export v${MONACO_VERSION} ==="
echo "Tenant: ${tenantId}"
echo ""

# --- Detect platform ---
OS=$(uname -s)
ARCH=$(uname -m)

case "${OS}-${ARCH}" in
    Darwin-arm64) platform="darwin-arm64" ;;
    Darwin-x86_64) platform="darwin-amd64" ;;
    Linux-x86_64) platform="linux-amd64" ;;
    Linux-aarch64) platform="linux-arm64" ;;
    Linux-i386) platform="linux-386" ;;
    *)
        echo "Error: Unsupported platform ${OS}-${ARCH}"
        exit 1
        ;;
esac

```

```

echo "Platform: ${platform}"

# --- Download Monaco ---
binary_url="https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/download/v${MONACO_VERSION}/monaco-${platform}"
checksum_url="${binary_url}.sha256"

echo "Downloading Monaco v${MONACO_VERSION}..."
curl -sL -o "monaco-${platform}" "${binary_url}"
curl -sL -o monaco_checksum "${checksum_url}"

echo "Verifying checksum..."
if shasum --quiet -c monaco_checksum 2>/dev/null; then
    echo "Checksum verified"
else
    echo "Checksum verification failed"
    rm -f "monaco-${platform}" monaco_checksum
    exit 1
fi

mv "monaco-${platform}" monaco
chmod +x monaco
echo "Monaco v${MONACO_VERSION} ready"
echo ""

# --- Revoke the export token on exit (success or failure) ---
MONACO_TOKEN_ID=""
revoke_export_token() {
    if [ -n "${MONACO_TOKEN_ID}" ] && [ "${MONACO_TOKEN_ID}" != "null" ]; then
        echo "Revoking temporary export token..."
        curl -s -o /dev/null -w "DELETE apiTokens -> HTTP %{http_code}\n" \
-X DELETE \

"https://${tenantId}.live.dynatrace.com/api/v2/apiTokens/${MONACO_TOKEN_ID}" \
-H "Authorization: Api-Token ${ENV_TOKEN}" \
|| echo "Revoke failed: delete token ${MONACO_TOKEN_ID} manually"
    fi
}
trap revoke_export_token EXIT

# --- Create short-lived export token (read scopes only, expires in 24 h) ---
echo "Creating temporary export token..."
expiry=$(date -u -v+1d +%Y-%m-%dT%H:%M:%S 2>/dev/null || date -u -d '+1 day' +%Y-%m-%dT%H:%M:%S)
token_body=$(jq -n --arg exp "${expiry}" '{
    name: "s2s-monaco-export-temp",

```

```

    expirationDate: $exp,
    scopes: [
      "attacks.read",
      "entities.read",
      "extensionConfigurations.read",
      "extensionEnvironment.read",
      "extensions.read",
      "geographicRegions.read",
      "javaScriptMappingFiles.read",
      "networkZones.read",
      "settings.read",
      "slo.read",
      "syntheticExecutions.read",
      "syntheticLocations.read",
      "DataExport",
      "ReadConfig",
      "ReadSyntheticData"
    ]
  }')

token_response=$(curl -s -X POST
"https://${tenantId}.live.dynatrace.com/api/v2/apiTokens" \
-H "accept: application/json; charset=utf-8" \
-H "Content-Type: application/json; charset=utf-8" \
-H "Authorization: Api-Token ${ENV_TOKEN}" \
-d "${token_body}")
MONACO_TOKEN=$(echo "${token_response}" | jq -r '.token // empty')
MONACO_TOKEN_ID=$(echo "${token_response}" | jq -r '.id // empty')

if [ -z "${MONACO_TOKEN}" ]; then
  echo "  Failed to create export token. Check ENV_TOKEN permissions
(apiTokens.write)."
  rm -f monaco monaco_checksum
  exit 1
fi
export MONACO_TOKEN
echo "  Export token created (expires ${expiry} UTC)"
echo ""

# --- Create manifest ---
cat &gt; manifest.yaml &lt;&lt;EOF
manifestVersion: 1.0

projects:
- name: saas
  path: saas/${tenantId}

environmentGroups:

```

```

- name: saas
  environments:
    - name: ${tenantId}
      url:
        value: https://${tenantId}.live.dynatrace.com
      auth:
        token:
          name: MONACO_TOKEN
EOF

# --- Run Monaco download ---
echo "Running Monaco download..."
echo "  This may take several minutes depending on configuration volume."
echo ""
./monaco download --environment "${tenantId}" --output-folder "${tenantId}"
echo ""

# The export token is no longer needed: revoke it now (the EXIT trap covers
failures)
revoke_export_token
MONACO_TOKEN_ID=""
unset MONACO_TOKEN
echo ""

# --- Package in SaaS Upgrade Assistant format ---
datetime=$(date +"%Y-%m-%d_%H-%M-%S")
directory_name="configurationExport-${datetime}"
mkdir -p "${directory_name}/export"

current_timestamp=$((date +%s) * 1000)

cat > "${directory_name}/exportMetadata.json" &&&EOF
{
  "clusterUuid": "${tenantId}",
  "productVersion": "1.305.0.20260331-000000",
  "monacoVersion": "${MONACO_VERSION}",
  "exportTimestamp": "${current_timestamp}",
  "environments": [
    {
      "name": "${tenantId}",
      "uuid": "${tenantId}"
    }
  ]
}
EOF

mv "${tenantId}"/* "${directory_name}/export/" 2>/dev/null || true

```

```
# --- Create archive ---
tar -czf "${directory_name}.tar.gz" "${directory_name}"
echo "Archive created: ${directory_name}.tar.gz"

# --- Summary ---
echo ""
echo "=== Export Summary ==="
echo "Tenant:    ${tenantId}"
echo "Monaco:    v${MONACO_VERSION}"
echo "Output:    ${directory_name}.tar.gz"
echo "Format:    SaaS Upgrade Assistant compatible"
echo ""

config_count=$(find "${directory_name}/export" -name "*.json" -o -name
"*.yaml" 2>/dev/null | wc -l | tr -d ' ')
echo "Configs exported: ~${config_count} files"
echo ""

echo "Next steps:"
echo "  1. Upload ${directory_name}.tar.gz to SaaS Upgrade Assistant on target
tenant"
echo "  2. Review configuration preview in SUA"
echo "  3. Deploy in waves per S2S-05 deployment order"
echo ""

# --- Cleanup ---
rm -f monaco monaco_checksum manifest.yaml
rm -rf "${directory_name}"

echo "Done. Temporary files cleaned up (archive preserved)."
```

3. Monaco Configuration Export (PowerShell)

Windows equivalent of the Bash script above. Produces the same `.tar.gz` archive for upload to the SaaS Upgrade Assistant.

Usage:

```
$env:ENV_TOKEN = "dt0c01.XXXX..."
.\Monaco-Export-Migration.ps1 -TenantId abc12345
```

Script:

```

#Requires -Version 5.1
<#
.SYNOPSIS
    Monaco Configuration Export for SaaS Upgrade Assistant
.DESCRIPTION
    Downloads Monaco, exports all configuration from a source tenant,
    and packages the result as a .tar.gz archive compatible with the
    SaaS Upgrade Assistant.
.PARAMETER TenantId
    The Dynatrace tenant identifier (e.g., abc12345)
.EXAMPLE
    $env:ENV_TOKEN = "dt0c01.XXXX..."
    .\Monaco-Export-Migration.ps1 -TenantId abc12345
#>
param(
    [Parameter(Mandatory)]
    [string]$TenantId
)

$ErrorActionPreference = "Stop"
$MonacoVersion = "2.30.0" # pinned; check the releases page before reuse

# --- Validate token ---
if (-not $env:ENV_TOKEN) {
    Write-Error "ENV_TOKEN environment variable is not set.`nCreate a token
with apiTokens.write scope at:`n
https://$TenantId.live.dynatrace.com/#settings/integration/apikeys"
}

Write-Host "=== Monaco Export v$MonacoVersion ==="
Write-Host "Tenant: $TenantId"
Write-Host ""

# --- Download Monaco (Windows amd64) ---
$platform = "windows-amd64"
$binaryUrl = "https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/download/v$MonacoVersion/monaco-$platform.exe"
$checksumUrl = "$binaryUrl.sha256"

Write-Host "Downloading Monaco v$MonacoVersion..."
Invoke-WebRequest -Uri $binaryUrl -OutFile "monaco.exe" -UseBasicParsing
Invoke-WebRequest -Uri $checksumUrl -OutFile "monaco_checksum" -
UseBasicParsing

Write-Host "Verifying checksum..."
$expectedHash = (Get-Content monaco_checksum -Raw).Split()[0].Trim()
$actualHash = (Get-FileHash -Algorithm SHA256 monaco.exe).Hash.ToLower()

```

```

if ($actualHash -ne $expectedHash) {
    Remove-Item -Force monaco.exe, monaco_checksum
    Write-Error "Checksum verification failed"
}
Write-Host "    Checksum verified"
Write-Host "    Monaco v$MonacoVersion ready"
Write-Host ""

# --- Create short-lived export token (read scopes only, expires in 24 h) ---
Write-Host "Creating temporary export token..."
$expiry = (Get-Date).ToUniversalTime().AddDays(1).ToString("yyyy-MM-
ddTHH:mm:ss")
$tokenBody = @{
    name          = "s2s-monaco-export-temp"
    expirationDate = $expiry
    scopes        = @(
        "attacks.read", "entities.read", "extensionConfigurations.read",
        "extensionEnvironment.read", "extensions.read",
        "geographicRegions.read",
        "javascriptMappingFiles.read", "networkZones.read", "settings.read",
        "slo.read", "syntheticExecutions.read", "syntheticLocations.read",
        "DataExport", "ReadConfig", "ReadSyntheticData"
    )
} | ConvertTo-Json

$headers = @{
    "Authorization" = "Api-Token $($env:ENV_TOKEN)"
    "Content-Type"  = "application/json; charset=utf-8"
}

$response = Invoke-RestMethod `
    -Uri "https://$TenantId.live.dynatrace.com/api/v2/apiTokens" `
    -Method Post -Headers $headers -Body $tokenBody

if (-not $response.token) {
    Remove-Item -Force monaco.exe, monaco_checksum -ErrorAction
    SilentlyContinue
    Write-Error "Failed to create export token. Check ENV_TOKEN permissions."
}
$env:MONACO_TOKEN = $response.token
$tokenId = $response.id
Write-Host "    Export token created (expires $expiry UTC)"
Write-Host ""

# --- Create manifest ---
@"
manifestVersion: 1.0

```

```

projects:
- name: saas
  path: saas/$TenantId

environmentGroups:
- name: saas
  environments:
- name: $TenantId
  url:
    value: https://$TenantId.live.dynatrace.com
  auth:
    token:
      name: MONACO_TOKEN
"@ | Set-Content -Path "manifest.yaml" -Encoding UTF8

# --- Run Monaco download ---
Write-Host "Running Monaco download..."
Write-Host " This may take several minutes depending on configuration
volume."
Write-Host ""
.\monaco.exe download --environment $TenantId --output-folder $TenantId
$downloadExit = $LASTEXITCODE
Write-Host ""

# --- Revoke the export token (also when the download failed) ---
Write-Host "Revoking temporary export token..."
try {
  Invoke-RestMethod `
    -Uri "https://$TenantId.live.dynatrace.com/api/v2/apiTokens/$tokenId"
    -Method Delete -Headers $headers | Out-Null
  Write-Host " Export token revoked"
} catch {
  Write-Warning "Revoke failed: delete token $tokenId manually (it expires
$expiry UTC regardless)"
}
Remove-Item Env:\MONACO_TOKEN -ErrorAction SilentlyContinue
if ($downloadExit -ne 0) {
  Write-Error "monaco download failed with exit code $downloadExit"
}
Write-Host ""

# --- Package in SaaS Upgrade Assistant format ---
$datetime = Get-Date -Format "yyyy-MM-dd_HH-mm-ss"
$directoryName = "configurationExport-$datetime"
New-Item -ItemType Directory -Path "$directoryName\export" -Force | Out-Null

$currentTimeStamp = [long]([DateTimeOffset]::UtcNow.ToUnixTimeMilliseconds())

```

```

@"
{
  "clusterUuid": "$TenantId",
  "productVersion": "1.305.0.20260331-000000",
  "monacoVersion": "$MonacoVersion",
  "exportTimestamp": "$currentTimestamp",
  "environments": [
    {
      "name": "$TenantId",
      "uuid": "$TenantId"
    }
  ]
}
"@ | Set-Content -Path "$directoryName\exportMetadata.json" -Encoding UTF8

if (Test-Path $TenantId) {
  Get-ChildItem -Path $TenantId | Move-Item -Destination
"$directoryName\export\" -Force
}

# --- Create tar.gz archive (Windows 10+ includes tar.exe) ---
tar -czf "$directoryName.tar.gz" $directoryName
Write-Host "Archive created: $directoryName.tar.gz"

# --- Summary ---
Write-Host ""
Write-Host "=== Export Summary ==="
Write-Host "Tenant:    $TenantId"
Write-Host "Monaco:    v$MonacoVersion"
Write-Host "Output:    $directoryName.tar.gz"
Write-Host "Format:    SaaS Upgrade Assistant compatible"
Write-Host ""

$configCount = (Get-ChildItem -Path "$directoryName\export" -Recurse -Include
*.json, *.yaml).Count
Write-Host "Configs exported: ~$configCount files"
Write-Host ""

Write-Host "Next steps:"
Write-Host " 1. Upload $directoryName.tar.gz to SaaS Upgrade Assistant on
target tenant"
Write-Host " 2. Review configuration preview in SUA"
Write-Host " 3. Deploy in waves per S2S-05 deployment order"
Write-Host ""

# --- Cleanup ---
Remove-Item -Force monaco.exe, monaco_checksum, manifest.yaml -ErrorAction

```

```
SilentlyContinue
Remove-Item -Recurse -Force $directoryName, $TenantId -ErrorAction
SilentlyContinue

Write-Host "Done. Temporary files cleaned up (archive preserved)."
```

Note: Windows 10 version 1803+ and Windows 11 include `tar.exe` natively. On older systems, install [7-Zip](#) and replace the `tar` line with:

```
powershell &gt; & "C:\Program Files\7-Zip\7z.exe" a -ttar
"$directoryName.tar" $directoryName & & "C:\Program Files\7-
Zip\7z.exe" a -tgzip "$directoryName.tar.gz" "$directoryName.tar" & &
Remove-Item "$directoryName.tar" & &
```

4. Usage Notes

Bash (macOS / Linux / WSL)

- Save the Bash script to a file (e.g., `monaco-export-migration.sh`) and run `chmod +x monaco-export-migration.sh`
- Requires `curl`, `jq`, `shasum`, and `tar`
- Tested on macOS (Apple Silicon + Intel), Ubuntu 22.04, and WSL2

PowerShell (Windows)

- Save the PowerShell script to a file (e.g., `Monaco-Export-Migration.ps1`)
- If execution policy blocks the script, run: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
- Requires Windows 10 version 1803+ or Windows 11 (for native `tar.exe`)

Both Scripts

- The script downloads Monaco automatically — no pre-installation required
- A short-lived export token (`s2s-monaco-export-temp`) is created with read scopes only, expires after 24 hours, and is revoked as soon as the download finishes — also when it fails

- Monaco is pinned to v2.30.0 (released 09/23/2026); check the [releases page](#) before reuse and bump `MONACO_VERSION` / `$MonacoVersion`
- Output is a `.tar.gz` archive compatible with the SaaS Upgrade Assistant
- The `.tar.gz` format is **required** by SUA — `.zip` archives are not accepted
- Upload the archive to the target tenant via the SaaS Upgrade Assistant app
- Deploy in waves per the deployment order in **S2S-05: Execute**

Multi-Source Consolidation

When consolidating multiple source tenants into a single target, run the script once per source tenant:

```
# Source 1
ENV_TOKEN=$SOURCE1_TOKEN ./monaco-export-migration.sh tenant-id-1

# Source 2
ENV_TOKEN=$SOURCE2_TOKEN ./monaco-export-migration.sh tenant-id-2
```

Import Source 1 first, validate, then import Source 2. Sequential migration isolates issues to a single source at a time. See **S2S-02** for the multi-source consolidation pattern.

5. Post-Export Workflow

After running the export script:

Step	Action	Reference
1	Upload <code>.tar.gz</code> to SaaS Upgrade Assistant on target tenant	Target tenant → Apps → SaaS Upgrade Assistant
2	Review configuration preview in SUA	SUA shows all config items; select what to deploy
3	Audit for entity ID references	Search exported config for <code>HOST-</code> , <code>SERVICE-</code> , <code>PROCESS_GROUP-</code> patterns
4	Remap entity IDs to tag-based filters	See S2S-05 §4: Entity ID Remapping
5	Deploy in waves per deployment order	See S2S-05 §1: Configuration Deployment Order

Step	Action	Reference
6	Validate data flow after each wave	See S2S-05 §8: Wave Execution and Validation

When NOT to Use These Scripts

Scenario	Use Instead
You only need specific config types	<code>monaco download --only-settings , --settings-schema <schema> or --api <api></code>
You want ongoing config management	Terraform with state management
Target is a Managed environment	SaaS Upgrade Assistant has its own export workflow
You need IAM migration	Terraform — Monaco cannot manage IAM

Script vs. Manual Monaco

Feature	Script	Manual <code>monaco download</code>
Monaco installation	Automatic (downloads + verifies checksum)	Manual pre-installation required
Export token	Auto-created with read scopes, 24 h expiry, revoked after download	Manual token creation and revocation
SUA-compatible packaging	Automatic <code>.tar.gz</code> with <code>exportMetadata.json</code>	Manual packaging required
Cleanup	Automatic (removes temp files)	Manual cleanup
Cross-platform	Bash + PowerShell versions	Single platform

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.